Faster Deterministic Streaming Vertex Coloring

Shiri Chechik[1] **Hongyi Chen**[2] Tianyi Zhang[2]

[1]Tel Aviv University

[2]Nanjing University

ICALP 2026

Vertex Coloring in Graph Streams

- **Input:** an undirected graph $G = (V, E)$ with n vertices and maximum degree Δ.

Vertex Coloring in Graph Streams

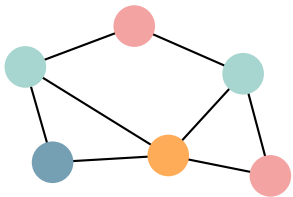
- **Input:** an undirected graph $G = (V, E)$ with n vertices and maximum degree Δ.
- **Stream access:** edges arrive one-by-one; the algorithm may scan the stream several times.

Vertex Coloring in Graph Streams

- **Input:** an undirected graph $G = (V, E)$ with n vertices and maximum degree Δ.
- **Stream access:** edges arrive one-by-one; the algorithm may scan the stream several times.
- **Semi-streaming space:** only $\widetilde{O}(n)$ bits.

Vertex Coloring in Graph Streams

- **Input:** an undirected graph $G = (V, E)$ with n vertices and maximum degree Δ.
- **Stream access:** edges arrive one-by-one; the algorithm may scan the stream several times.
- **Semi-streaming space:** only $\widetilde{O}(n)$ bits.
- **Goal:** output a proper vertex coloring using few colors and few passes.



Proper coloring:
adjacent vertices get different colors.

Background: Randomized Algorithms

- With randomness, one pass already suffices for strong vertex colorings in semi-streaming space.

Result	Colors	Passes	Comment
Bera, Ghosh '18	$O(\Delta)$	1	
Assadi, Chen, Khanna '19	$\Delta + 1$	1	
Bera, Chakrabarti, Ghosh '20	$(1 + o(1))\kappa$	1	κ is the degeneracy.
Assadi, Kumar, Mittal '22	Δ	1	Brooks' theorem.

Background: Adversarially Robust Algorithms

- The adversary may adaptively choose the next edge after seeing the transcript so far.
- The algorithm must remain correct with high probability at all times.
- Robust randomized algorithms are much closer in spirit to deterministic algorithms; a polynomial gap between lower and upper bounds remains.

Background: Adversarially Robust Algorithms

- The adversary may adaptively choose the next edge after seeing the transcript so far.
- The algorithm must remain correct with high probability at all times.
- Robust randomized algorithms are much closer in spirit to deterministic algorithms; a polynomial gap between lower and upper bounds remains.

Result	Colors	Passes	Comment
Chakrabarti, Ghosh, Stoeckl '22	$\Omega(\Delta^2)$	1	lower bound
Chakrabarti, Ghosh, Stoeckl '22	$O(\Delta^3)$	1	oracle randomness
Assadi, Chakrabarti, Ghosh, Stoeckl '23	$O(\Delta^{5/2})$	1	oracle randomness
Assadi, Chakrabarti, Ghosh, Stoeckl '23	$O(\Delta^3)$	1	randomness counted

Background: Deterministic Algorithms

- Deterministic one-pass algorithms require exponentially many colors.
- Therefore, for deterministic algorithms, pass complexity becomes the resource to optimize.

Result	Colors	Passes	Comment
Assadi, Chen, Sun '22	$\exp(\Delta^{\Omega(1)})$	1	lower bound
Assadi, Chen, Sun '22	$O(\Delta^2)$	2	
Assadi, Chen, Sun '22	$O(\Delta)$	$O(\log \Delta)$	
Assadi, Chakrabarti, Ghosh, Stoeckl '23	$\Delta + 1$	$O(\log \Delta \log\log \Delta)$	

Questions in the Deterministic Landscape

Upper-bound questions

- What is the best palette size achievable in **2 passes**?
- How many passes are needed for $(\Delta+1)$-**coloring**, or for $O(\Delta)$-**coloring**?

Lower-bound questions

Questions in the Deterministic Landscape

Upper-bound questions

- What is the best palette size achievable in **2 passes**?
- How many passes are needed for $(\Delta+1)$-**coloring**, or for $O(\Delta)$-**coloring**?

Lower-bound questions

- No nontrivial multi-pass lower bounds are known, even for **2 passes**.
- Can we prove pass lower bounds for $O(\Delta)$ **colors**, or even for $(\Delta+1)$ **colors**?

Questions in the Deterministic Landscape

Upper-bound questions

- What is the best palette size achievable in **2 passes**?
- How many passes are needed for $(\Delta+1)$-**coloring**, or for $O(\Delta)$-**coloring**?

Lower-bound questions

- No nontrivial multi-pass lower bounds are known, even for **2 passes**.
- Can we prove pass lower bounds for $O(\Delta)$ **colors**, or even for $(\Delta+1)$ **colors**?

This talk

We make progress on one upper-bound question: deterministic $O(\Delta)$-coloring in $O(\sqrt{\log \Delta})$ passes.

Our Result

Main theorem

There is a deterministic semi-streaming algorithm that computes a $O(\Delta)$-coloring using

$$O\left(\sqrt{\log \Delta}\right) \quad \text{passes and} \quad \widetilde{O}(n) \text{ bits of space.}$$

Our Result

Main theorem

There is a deterministic semi-streaming algorithm that computes a $O(\Delta)$-coloring using

$$O\left(\sqrt{\log \Delta}\right) \quad \text{passes and} \quad \widetilde{O}(n) \text{ bits of space.}$$

Takeaway

This is the first deterministic streaming $O(\Delta)$-coloring with sublogarithmic pass complexity.

Random Palette Sampling [ACS'22]

- In one phase: reduce the number of uncolored vertices by a constant factor.

Random Palette Sampling [ACS'22]

- In one phase: reduce the number of uncolored vertices by a constant factor.
- Repeating this for $O(\log \Delta)$ phases leaves only $O(n/\Delta)$ uncolored vertices.

Random Palette Sampling [ACS'22]

- In one phase: reduce the number of uncolored vertices by a constant factor.
- Repeating this for $O(\log \Delta)$ phases leaves only $O(n/\Delta)$ uncolored vertices.

Random phase

- Choose $c(v)$ uniformly from the full palette $[10\Delta]$ for every uncolored vertex v.
- If $c(v)$ creates no conflict with any neighbor, permanently give v this color.
- By a union bound,

$$\Pr[v \text{ has a conflict}] \leq \frac{\Delta}{10\Delta} \leq \frac{1}{10}.$$

- Thus a constant fraction of the uncolored vertices is colored in expectation.

Hash Derandomization [ACS'22]

- A hash function $h : V \to [10\Delta]$ is a candidate tentative color assignment.
- Instead of choosing one random assignment, enumerate a small family of candidates.

Hash Derandomization [ACS'22]

- A hash function $h : V \to [10\Delta]$ is a candidate tentative color assignment.
- Instead of choosing one random assignment, enumerate a small family of candidates.

Derandomization

- Use a near-universal hash family[a] $\mathcal{H} = \{h : V \to [10\Delta]\}$ with $|\mathcal{H}| = O(n)$.
- First pass: for every $h \in \mathcal{H}$, count the # of monochromatic edges; choose the best $h^\star$.
- Second pass: identify vertices incident to monochromatic edges under $h^\star$, and color all other uncolored vertices.

[a] This works because the analysis only needs a hash function with few total conflicts.

Bottleneck

- Suppose $|U| = \varepsilon n$ vertices remain uncolored.

Bottleneck

- Suppose $|U| = \varepsilon n$ vertices remain uncolored.
- Full-palette sampling has two sources of conflict:

$$\underbrace{O(\varepsilon\Delta) \cdot O(1/\Delta)}_{\text{uncolored neighbors} = O(\varepsilon)} \quad + \quad \underbrace{O(\Delta) \cdot O(1/\Delta)}_{\text{colored neighbors} = O(1)} \; .$$

Bottleneck

- Suppose $|U| = \varepsilon n$ vertices remain uncolored.
- Full-palette sampling has two sources of conflict:

$$\underbrace{O(\varepsilon\Delta) \cdot O(1/\Delta)}_{\text{uncolored neighbors} = O(\varepsilon)} \quad + \quad \underbrace{O(\Delta) \cdot O(1/\Delta)}_{\text{colored neighbors} = O(1)} \; .$$

- The second term does *not* shrink with $|U|$, so each phase only gives constant-factor progress.

Bottleneck

- Suppose $|U| = \varepsilon n$ vertices remain uncolored.
- Full-palette sampling has two sources of conflict:

$$\underbrace{O(\varepsilon\Delta) \cdot O(1/\Delta)}_{\text{uncolored neighbors} = O(\varepsilon)} \quad + \quad \underbrace{O(\Delta) \cdot O(1/\Delta)}_{\text{colored neighbors} = O(1)} \; .$$

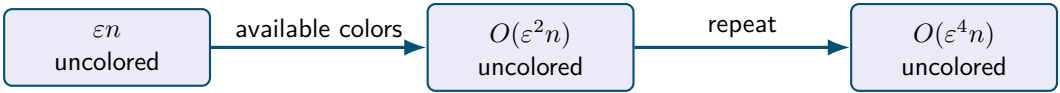
- The second term does *not* shrink with $|U|$, so each phase only gives constant-factor progress.
- Dream: sample from $\mathrm{Avail}(v)$ instead. Then the colored-neighbor term disappears.

Bottleneck

- Suppose $|U| = \varepsilon n$ vertices remain uncolored.
- Full-palette sampling has two sources of conflict:

$$\underbrace{O(\varepsilon\Delta) \cdot O(1/\Delta)}_{\text{uncolored neighbors} = O(\varepsilon)} \quad + \quad \underbrace{O(\Delta) \cdot O(1/\Delta)}_{\text{colored neighbors} = O(1)} .$$

- The second term does *not* shrink with $|U|$, so each phase only gives constant-factor progress.
- Dream: sample from $\mathrm{Avail}(v)$ instead. Then the colored-neighbor term disappears.



Would suggest $O(\log\log\Delta)$ phases.

The Missing-Item Barrier

- The stream reveals the unavailable colors around each vertex.

The Missing-Item Barrier

- The stream reveals the unavailable colors around each vertex.
- But the algorithm wants a missing color.

The Missing-Item Barrier

- The stream reveals the unavailable colors around each vertex.
- But the algorithm wants a missing color.
- More precisely: given a stream $x_1, \ldots, x_N \in [M]$, output any $y \in [M] \setminus \{x_1, \ldots, x_N\}$.

The Missing-Item Barrier

- The stream reveals the unavailable colors around each vertex.
- But the algorithm wants a missing color.
- More precisely: given a stream $x_1, \ldots, x_N \in [M]$, output any $y \in [M] \setminus \{x_1, \ldots, x_N\}$.
- Every deterministic one-pass algorithm needs $\widetilde{\Omega}(N)$ space Stoeckl '23.

The Missing-Item Barrier

- The stream reveals the unavailable colors around each vertex.
- But the algorithm wants a missing color.
- More precisely: given a stream $x_1, \ldots, x_N \in [M]$, output any $y \in [M] \setminus \{x_1, \ldots, x_N\}$.
- Every deterministic one-pass algorithm needs $\widetilde{\Omega}(N)$ space Stoeckl '23.
- Here N can be $\Theta(\Delta)$ for each uncolored vertex.

Method: Vertex-Color Partition

- We want to partition both vertices and colors:

$$(B_1, C_1), \ldots, (B_\ell, C_\ell).$$

Method: Vertex-Color Partition

- We want to partition both vertices and colors:

$$(B_1, C_1), \ldots, (B_\ell, C_\ell).$$

- Each B_i should be colorable using *only* colors from C_i.

Method: Vertex-Color Partition

- We want to partition both vertices and colors:

$$(B_1, C_1), \ldots, (B_\ell, C_\ell).$$

- Each B_i should be colorable using *only* colors from C_i.
- Sufficient local slack:

$$|\mathrm{Avail}(v) \cap C_i| > |N(v) \cap B_i| \qquad \text{for every } v \in B_i.$$

Method: Vertex-Color Partition

- We want to partition both vertices and colors:

$$(B_1, C_1), \ldots, (B_\ell, C_\ell).$$

- Each B_i should be colorable using *only* colors from C_i.
- Sufficient local slack:

$$|\text{Avail}(v) \cap C_i| > |N(v) \cap B_i| \qquad \text{for every } v \in B_i.$$

- If both B_i and C_i are small, store local edges and available lists, then color offline.

Method: Vertex-Color Partition

- We follow the recursive partition paradigm from distributed coloring [CDP'20].

Method: Vertex-Color Partition

- We follow the recursive partition paradigm from distributed coloring [CDP'20].
- Refine each pair: $(B, C) \mapsto (B_1, C_1), \ldots, (B_\ell, C_\ell)$.

Method: Vertex-Color Partition

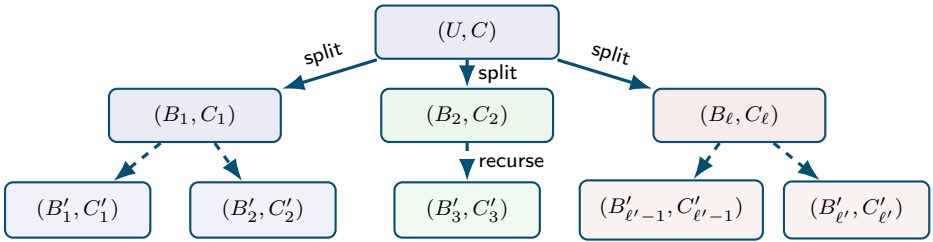
- We follow the recursive partition paradigm from distributed coloring [CDP'20].
- Refine each pair: $(B, C) \mapsto (B_1, C_1), \ldots, (B_\ell, C_\ell)$.
- Each level shrinks both the vertex side and the color side.

Method: Vertex-Color Partition

- We follow the recursive partition paradigm from distributed coloring [CDP'20].
- Refine each pair: $(B, C) \mapsto (B_1, C_1), \ldots, (B_\ell, C_\ell)$.
- Each level shrinks both the vertex side and the color side.

Method: Vertex-Color Partition

- We follow the recursive partition paradigm from distributed coloring [CDP'20].
- Refine each pair: $(B, C) \mapsto (B_1, C_1), \ldots, (B_\ell, C_\ell)$.
- Each level shrinks both the vertex side and the color side.



One Refinement Round

1. For a current pair (B, C), choose $h_1 \in \mathcal{H}_1$ for vertices and $h_2 \in \mathcal{H}_2$ for colors.

One Refinement Round

1. For a current pair (B, C), choose $h_1 \in \mathcal{H}_1$ for vertices and $h_2 \in \mathcal{H}_2$ for colors.
2. These induce matched children

$$B_j = \{v \in B : h_1(v) = j\}, \qquad C_j = \{c \in C : h_2(c) = j\}.$$

One Refinement Round

1. For a current pair (B, C), choose $h_1 \in \mathcal{H}_1$ for vertices and $h_2 \in \mathcal{H}_2$ for colors.
2. These induce matched children

$$B_j = \{v \in B : h_1(v) = j\}, \qquad C_j = \{c \in C : h_2(c) = j\}.$$

3. A vertex $v \in B$, with $j = h_1(v)$, votes for (h_1, h_2) if

$$|\mathrm{Avail}(v) \cap C_j| > |N(v) \cap B_j|.$$

Here $\mathrm{Avail}(v)$ means colors not used by already-colored neighbors.

One Refinement Round

1. For a current pair (B, C), choose $h_1 \in \mathcal{H}_1$ for vertices and $h_2 \in \mathcal{H}_2$ for colors.
2. These induce matched children

$$B_j = \{v \in B : h_1(v) = j\}, \qquad C_j = \{c \in C : h_2(c) = j\}.$$

3. A vertex $v \in B$, with $j = h_1(v)$, votes for (h_1, h_2) if

$$|\mathrm{Avail}(v) \cap C_j| > |N(v) \cap B_j|.$$

   Here $\mathrm{Avail}(v)$ means colors not used by already-colored neighbors.
4. Choose a pair with the fewest failed vertices; by averaging, a good pair has only a small failed set.

Derandomization via $O(1)$-wise independence

- In congested clique algorithms, recursive partitions can be derandomized using $O(1)$-wise independent hash families.

Derandomization via $O(1)$-wise independence

- In congested clique algorithms, recursive partitions can be derandomized using $O(1)$-wise independent hash families.
- Derandomization via conditional expectation is feasible with global communication.
- In streaming, testing a pair (h_1, h_2) requires counters for every

$$v \in U, \qquad (h_1, h_2) \in \mathcal{H}_1 \times \mathcal{H}_2.$$

Derandomization via $O(1)$-wise independence

- In congested clique algorithms, recursive partitions can be derandomized using $O(1)$-wise independent hash families.
- Derandomization via conditional expectation is feasible with global communication.
- In streaming, testing a pair (h_1, h_2) requires counters for every

$$v \in U, \qquad (h_1, h_2) \in \mathcal{H}_1 \times \mathcal{H}_2.$$

- Exact k-wise families are too large to enumerate:

$$|\mathcal{H}_1^{\text{exact}}| = n^{O(k)}, \qquad |\mathcal{H}_2^{\text{exact}}| = \Delta^{O(k)}.$$

Derandomization via $O(1)$-wise independence

- In congested clique algorithms, recursive partitions can be derandomized using $O(1)$-wise independent hash families.
- Derandomization via conditional expectation is feasible with global communication.
- In streaming, testing a pair (h_1, h_2) requires counters for every

$$v \in U, \qquad (h_1, h_2) \in \mathcal{H}_1 \times \mathcal{H}_2.$$

- Exact k-wise families are too large to enumerate:

$$|\mathcal{H}_1^{\text{exact}}| = n^{O(k)}, \qquad |\mathcal{H}_2^{\text{exact}}| = \Delta^{O(k)}.$$

- So exact independence costs roughly

$$|U| \cdot n^{O(k)} \cdot \Delta^{O(k)}$$

counters, far beyond semi-streaming space.

Almost k-wise independence

- For $h : [A] \to [B]$, almost k-wise independence allows a small error:

$$\Pr[h(x_i) = y_i \ \forall i] = \frac{1}{B^k} \pm \varepsilon.$$

Almost k-wise independence

- For $h : [A] \to [B]$, almost k-wise independence allows a small error:

$$\Pr[h(x_i) = y_i \ \forall i] = \frac{1}{B^k} \pm \varepsilon.$$

- Compared with exact independence, we trade a small error for a much smaller family:

$$\underbrace{\max\{A, B\}^{O(k)}}_{\text{exact } k\text{-wise}} \qquad \rightsquigarrow \qquad \underbrace{B^{O(k)} \operatorname{poly}(1/\varepsilon, \log A)}_{\text{almost } k\text{-wise}}.$$

Almost k-wise independence

- For $h : [A] \to [B]$, almost k-wise independence allows a small error:

$$\Pr[h(x_i) = y_i \ \forall i] = \frac{1}{B^k} \pm \varepsilon.$$

- Compared with exact independence, we trade a small error for a much smaller family:

$$\underbrace{\max\{A, B\}^{O(k)}}_{\text{exact } k\text{-wise}} \qquad \rightsquigarrow \qquad \underbrace{B^{O(k)} \operatorname{poly}(1/\varepsilon, \log A)}_{\text{almost } k\text{-wise}}.$$

- The concentration is slightly weaker, but the moment bound still works for our estimates.

Almost k-wise independence

- For $h : [A] \to [B]$, almost k-wise independence allows a small error:

$$\Pr[h(x_i) = y_i \ \forall i] = \frac{1}{B^k} \pm \varepsilon.$$

- Compared with exact independence, we trade a small error for a much smaller family:

$$\underbrace{\max\{A, B\}^{O(k)}}_{\text{exact } k\text{-wise}} \qquad \rightsquigarrow \qquad \underbrace{B^{O(k)} \operatorname{poly}(1/\varepsilon, \log A)}_{\text{almost } k\text{-wise}}.$$

- The concentration is slightly weaker, but the moment bound still works for our estimates.
- In one refinement, let M be the hash range. With $k = 10$ and suitable ε,

$$|\mathcal{H}_1| = M^{O(1)} \operatorname{polylog} n, \qquad |\mathcal{H}_2| = M^{O(1)} \operatorname{polylog} \Delta.$$

Almost k-wise independence

- For $h : [A] \to [B]$, almost k-wise independence allows a small error:

$$\Pr[h(x_i) = y_i \ \forall i] = \frac{1}{B^k} \pm \varepsilon.$$

- Compared with exact independence, we trade a small error for a much smaller family:

$$\underbrace{\max\{A, B\}^{O(k)}}_{\text{exact } k\text{-wise}} \qquad \rightsquigarrow \qquad \underbrace{B^{O(k)} \operatorname{poly}(1/\varepsilon, \log A)}_{\text{almost } k\text{-wise}}.$$

- The concentration is slightly weaker, but the moment bound still works for our estimates.
- In one refinement, let M be the hash range. With $k = 10$ and suitable ε,

$$|\mathcal{H}_1| = M^{O(1)} \operatorname{polylog} n, \qquad |\mathcal{H}_2| = M^{O(1)} \operatorname{polylog} \Delta.$$

- First use Assadi, Chen, Sun '22 to make $|U| \le n/M^C$, where C dominates the hidden exponent above. Then

$$|U| \cdot |\mathcal{H}_1| \cdot |\mathcal{H}_2| = \widetilde{O}(n).$$

Prune Frequent Colors

- To evaluate a vote, we need a lower bound on $|\mathrm{Avail}(v) \cap C_j|$.

Prune Frequent Colors

- To evaluate a vote, we need a lower bound on $|\mathrm{Avail}(v) \cap C_j|$.
- The stream-visible lower bound counts *occurrences*:

$$|\mathrm{Avail}(v) \cap C_j| \geq |C_j| - |\{u \in N(v) : \phi(u) \in C_j\}|.$$

Prune Frequent Colors

- To evaluate a vote, we need a lower bound on $|\mathrm{Avail}(v) \cap C_j|$.
- The stream-visible lower bound counts *occurrences*:

$$|\mathrm{Avail}(v) \cap C_j| \geq |C_j| - |\{u \in N(v) : \phi(u) \in C_j\}|.$$

- This can be too pessimistic:

$$\phi(u_1) = \cdots = \phi(u_r) = c \quad \text{removes one color, but is charged } r \text{ times.}$$

Prune Frequent Colors

- To evaluate a vote, we need a lower bound on $|\mathrm{Avail}(v) \cap C_j|$.
- The stream-visible lower bound counts *occurrences*:

$$|\mathrm{Avail}(v) \cap C_j| \geq |C_j| - |\{u \in N(v) : \phi(u) \in C_j\}|.$$

- This can be too pessimistic:

$$\phi(u_1) = \cdots = \phi(u_r) = c \quad \text{removes one color, but is charged } r \text{ times.}$$

- Prune frequent neighbor colors into a small set F_v, and estimate availability inside $C_j \setminus F_v$.

Prune Frequent Colors

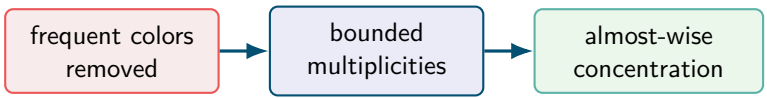
- To evaluate a vote, we need a lower bound on $|\mathrm{Avail}(v) \cap C_j|$.
- The stream-visible lower bound counts *occurrences*:

$$|\mathrm{Avail}(v) \cap C_j| \geq |C_j| - |\{u \in N(v) : \phi(u) \in C_j\}|.$$

- This can be too pessimistic:

$$\phi(u_1) = \cdots = \phi(u_r) = c \quad \text{removes one color, but is charged } r \text{ times.}$$

- Prune frequent neighbor colors into a small set F_v, and estimate availability inside $C_j \setminus F_v$.



Summary and Open Problems

This work

Deterministic $O(\Delta)$-coloring in $O(\sqrt{\log \Delta})$ passes and semi-streaming space.

Summary and Open Problems

This work

Deterministic $O(\Delta)$-coloring in $O(\sqrt{\log \Delta})$ passes and semi-streaming space.

Open questions

- Can we achieve $O(\Delta)$ colors, or even $(\Delta + 1)$ colors, in $O(\log \log \Delta)$ passes?
- Can we prove any nontrivial multi-pass lower bound?
- Can deterministic semi-streaming MIS beat $O(\log n)$ passes?

Thank you!

Questions?

Faster Deterministic Streaming Vertex Coloring